Práctica 1: Aprendizaje Supervisado

Izan Cuesta Corbí
DNI: 48804930B
Universidad de Alicante – Escuela Politécnica Superior
Aprendizaje Avanzado – Curso 2025/2026
Email: icc87@alu.ua.es

Resumen

Esta práctica se centra en la predicción del riesgo de ictus mediante un conjunto de datos clínicos altamente desbalanceado, donde menos del 5 % de los pacientes presentan la patología. La tarea de clasificación consiste en identificar correctamente a los individuos en riesgo a partir de variables como la edad, niveles de glucosa e hipertensión. Se han evaluado y comparado diversos algoritmos de aprendizaje supervisado: SVM, Árboles de Decisión, Random Forest, Gradient Boosting y AdaBoost. Para mejorar el rendimiento, se empleó una estrategia de optimización basada en GridSearchCV y validación cruzada, ajustando parámetros críticos como la profundidad de los árboles, la tasa de aprendizaje y, fundamentalmente, el balanceo de pesos de clase. Los resultados demuestran que, aunque el Random Forest obtiene el Accuracy más alto (95.42 %), este valor es engañoso debido al sesgo hacia la clase mayoritaria. El mejor modelo desde una perspectiva médica es AdaBoost, que logra un F1-Score de 0.8254 y un Accuracy en test de 0.7525. La clave de su éxito reside en su capacidad de maximizar el Recall (0.6190), permitiendo detectar a la mayoría de los pacientes con riesgo real a cambio de un aumento controlado en los falsos positivos.

Palabras clave: Aprendizaje supervisado, SVM, Árbol de Decisión, Random Forest, Gradient Boosting, Clasificación.

Índice

1. Dataset y Preprocesamiento **3**
1.1. Dataset . . . 3
1.2. Preprocesamiento . . . 3
1.3. Balanceo de clases . . . 3

2. Parte 1: Support Vector Machines **3**
2.1. Comparación de Kernels . . . 3
2.2. Optimización del Kernel RBF . . . 4
2.3. Optimización del Kernel Polinomial . . . 4
2.4. Comparación de Implementaciones (Kernel Lineal) . . . 5
2.5. Análisis de Vectores de Soporte . . . 6
2.6. Evaluación Final – SVM . . . 6

3. Parte 2: Árboles de Decisión **7**
3.1. Árbol sin Restricciones (Baseline) . . . 7
3.2. Efecto de la Profundidad Máxima . . . 8
3.3. Criterios de División . . . 8
3.4. Poda Previa: Grid Search de Hiperparámetros . . . 9
3.5. Poda Posterior: Cost-Complexity Pruning . . . 9
3.6. Importancia de Características . . . 10
3.7. Visualización del Árbol . . . 10
3.8. Evaluación Final – Árboles de Decisión . . . 11

4. Parte 3: Random Forest **12**
4.1. Random Forest por Defecto . . . 12
4.2. Efecto del Número de Estimadores . . . 12
4.3. Efecto de `max_features` . . . 12
4.4. Optimización mediante Grid Search . . . 13
4.5. Importancia de Características . . . 13
4.6. Extra Trees . . . 14
4.7. Evaluación Final – Random Forest . . . 14

5. Parte 4: Gradient Boosting **15**
5.1. Gradient Boosting por Defecto . . . 16
5.2. Efecto del Número de Estimadores y Learning Rate . . . 16
5.3. Efecto de `max_depth` . . . 17
5.4. Optimización mediante Grid Search . . . 17
5.5. AdaBoost . . . 17
5.6. Evaluación Final – Gradient Boosting . . . 18

6. Comparación y Evaluación Global **19**
6.1. Tabla Comparativa Final . . . 19
6.2. Análisis y Discusión Global . . . 20
6.3. Matriz de Confusión del Mejor Modelo . . . 20

7. Conclusiones **21**

1. Dataset y Preprocesamiento

1.1. Dataset

Dataset: *Stroke Prediction Dataset*

Tal y como se analizó y documentó exhaustivamente en la memoria de la **Práctica 0**[1], para este proyecto partimos del mismo conjunto de datos clínico destinado a la predicción del riesgo de ictus.

1.2. Preprocesamiento

- **Valores faltantes:** Únicamente detecté valores nulos en la variable bmi (Índice de Masa Corporal). La estrategia aplicada fue la imputación por la media de la columna para no perder registros, ya que representaban menos del 5 % del dataset.
- **Escalado:** Apliqué StandardScaler. Fundamental para modelos como SVM, ya que estandariza las características para que tengan media 0 y varianza 1, evitando que las variables con rangos grandes (como los niveles de glucosa) dominen sobre las pequeñas.
- **Codificación:** Utilicé One-Hot Encoding para las variables categóricas nominales (como gender, work_type, residence_type y smoking_status), creando columnas binarias para cada categoría.
- **División train/test:** La división se basa en un ratio del 80 % para entrenamiento y 20 % para test, y una estratificación basada en la variable objetivo (stroke) para asegurar que la baja proporción de casos de ictus se mantuviera idéntica en ambos conjuntos, manteniendo una semilla fija (random_state=42) para garantizar la reproducibilidad.

1.3. Balanceo de clases

Recordando la práctica anterior, para cada modelo que se cree en esta práctica, vamos a necesitar utilizar el parámetro class_weight='balanced', ya que nuestro dataset está extremadamente desbalanceado y como ya vimos en la práctica anterior, en caso de no balancearlo manualmente en el modelo los resultados saldrían sesgados y sin ningún tipo de interés médico, ya que se basaría en el porcentaje global para hacer las predicciones y no en las características individuales de cada paciente.

2. Parte 1: Support Vector Machines

2.1. Comparación de Kernels

Se entrenaron modelos SVM con los cuatro kernels principales usando los hiperparámetros por defecto y validación cruzada de 5 pliegues. Todos los modelos incorporan un `StandardScaler` dentro de un `Pipeline`.

Tabla 1: Rendimiento de SVM con distintos kernels (hiperparámetros por defecto)

Kernel	CV Acc	Test Acc	Prec	Recall	F1	N SV (%)
Linear	0.7473	0.7322	0.9454	0.7322	0.8118	1908 (48.6 %)
Poly (d=3)	0.8240	0.7994	0.9322	0.7994	0.8549	1735 (44.2 %)
RBF	0.7970	0.7872	0.9382	0.7872	0.8479	1739 (44.3 %)
Sigmoid	0.7025	0.6986	0.9397	0.6986	0.7883	1296 (33.0 %)

Con la configuración por defecto, el modelo **Poly (Polinómico)** obtiene el *Test Accuracy* más alto (74,89 %), seguido muy de cerca por el **Linear** (74,47 %). Ambos modelos presentan un rendimiento predictivo prácticamente idéntico y casi la misma cantidad de vectores de soporte utilizados.

En cuanto a la diferencia entre el *CV accuracy* y *test accuracy* en los 4 kernels es muy pequeña, en el caso del **Poly** siendo el menos ajustado con una diferencia del 0,0246, pero viendo ejemplos solo

[1] Cuesta Corbí, Izan. (2026). *Práctica 0: Flujo completo de Machine Learning*. Universidad de Alicante.

varían alguna centésima, lo que nos indica que no existe *overfitting* en estos modelos y generalizan bien en datos nunca vistos. Por otro lado, sí que observamos un caso de *underfitting* en el kernel **Sigmoid**, siendo el modelo con la precisión más baja (*test accuracy* no llega al 70 %), demostrando que con los parámetros por defecto es incapaz de capturar la relación real entre las variables de los pacientes y el riesgo de ictus.

En cuanto al número de vectores de soporte, podemos observar como no sigue una relación lineal directa con la complejidad del modelo. Los dos kernels más óptimos hasta el momento han sido **Poly** y **RBF**, ambos situándose en 1735-1739 SV ($\sim 44,2\,\%$ de los datos de *train*), sin embargo, ni reducir los SV como en el caso de **Sigmoid** (1296) ni aumentarlos como en **Linear** (1908) aseguran una mejora del *accuracy*.

2.2. Optimización del Kernel RBF

Se realizó una búsqueda en rejilla (*GridSearchCV*, 5-fold CV) sobre los hiperparámetros `C` y `gamma` del kernel RBF. Recuerda que `C` controla el trade-off margen/errores y `gamma` la influencia de cada muestra en la frontera de decisión.

Tabla 2: Exploración de hiperparámetros para SVM-RBF (selección representativa)

C	gamma	CV Acc	Test Acc	N SV	% Train	Tiempo (s)
0.1	0.001	0.7407	0.7424	3399	86.6 %	0.9760
0.1	scale	0.7224	0.7210	2243	57.1 %	0.6615
1	scale	0.7970	0.7872	1739	44.3 %	0.5753
1	0.1	0.8242	0.8024	1641	41.8 %	0.5150
10	0.01	0.8348	0.8220	2004	51.0 %	0.6222
10	0.1	0.8726	0.8574	1080	27.5 %	0.3723
100	0.001	0.7440	0.7301	1886	48.0 %	0.6646
100	0.1	0.8953	0.8870	765	19.5 %	0.3539
100	1	0.9325	0.9257	525	13.4 %	0.3760
Mejor	**1**	**0.9325**	**0.9257**	**525**	**13.4 %**	**0.3760**

Con una **C** pequeña, el modelo es más permisivo con los errores, lo que suele aumentar el número de vectores de soporte al incluir puntos dentro del margen. Con una **C** grande, el modelo intenta clasificar correctamente cada punto de entrenamiento, reduciendo el margen y disminuyendo el número de vectores de soporte (solo los muy cercanos a la frontera). Esto se puede ver fácilmente en la tabla, conforme vamos bajando aumentando **C**, el número de **SV** pasa de rondar entre los 2243 - 3513 cuando **C=0.1** hasta un rango entre 1886 - 525 cuando **C=100**. En cuanto a su relación con el *trade-off*, una **C** alta reduce el sesgo ajustándose mucho a los datos pero aumenta la varianza (riesgo de sobreajuste), en cambio, una **C** baja aumenta el sesgo pero mejora la generalización.

Para **Gamma**, cuando es muy alta la frontera de decisión se vuelve extremadamente compleja que capturando el ruido del *dataset* y creando islas alrededor de puntos específicos. Cuando es muy baja, la influencia de los puntos es muy amplia, lo que produce una frontera demasiado suave (casi lineal), incapaz de capturar la estructura no lineal de los datos de ictus. En la tabla, podemos observar como los valores de *CV* y *Test Accuracy* mejoran significativamente pasando de **gamma = 0.001** a **gamma = 1**. Por ejemplo, en los primeros casos de **C=0.1**, con **gamma=0.001** el *test accuracy* era de 0,7424, mientras que si le subimos la **gamma** a **1**, aumentaremos el *accuracy* hasta 0,8218, lo que supone una mejora del $0,0794$ en la precisión del test.

Una vez analizado todo esto, los mejores hiperparámetros para este kernel son **C=100** y **gamma=1**, dando un 0,9325 en el *CV accuracy* y un 0,9257 en el *test accuracy* (también significa el menor número de vectores de soporte de todo el análisis).

2.3. Optimización del Kernel Polinomial

De manera clara, el **grado 4** es el que mejor precisión da. Haciendo una breve comparación usando como ejemplo las ejecuciones con **C=10** y **gamma=0.1**, pasamos de valores de *test accuracy* desde

Tabla 3: Exploración de hiperparámetros para SVM Polinomial

degree	C	gamma	CV Acc	Test Acc	N SV
2	0.1	scale	0.7957	0.7790	2557
2	1	scale	0.7738	0.7648	2015
3	1	scale	0.8240	0.7994	1735
3	10	auto	0.8502	0.8299	1256
4	1	scale	0.8602	0.8259	1757
4	10	0.1	0.8854	0.8686	883
Mejor	**4**	**10**	**0.8854**	**0.8686**	**883**

grado=2 con 0,7617, a **grado=3** con 0,8411, hasta **grado=4** con 0,8686. Esto se debe a que conforme el grado aumenta, la complejidad del modelo crece exponencialmente, lo cual puede a llevarnos a un caso de *overfitting* en caso de que se aumentase más el grado, ya que se estaría intentando ajustar una frontera que pase por cada punto del *set* de entrenamiento, capturando el ruido y las excepciones en lugar de la tendencia general. En nuestro análisis, podemos considerar que el **grado 4** logra flexibilizar la frontera lo suficiente para adaptarse a la curvatura de los datos sin perder el control de basarse solo en los vistos.

Si comparamos estos resultados con los del kernel **RBF** calculado en el apartado anterior (ambos con sus mejores configuraciones), vemos que el **RBF** es superior con un 0,9257 frente al 0,8686 correspondiente al **Poly** en sus *tests accuracy*. Esto se debe a que el kernel **RBF** es más robusto al poder mapear los datos a un espacio de dimensiones infinitas, mientras que el **Poly** está limitado al grado seleccionado.

2.4. Comparación de Implementaciones (Kernel Lineal)

Scikit-learn ofrece tres implementaciones de SVM lineal con distinta formulación y coste computacional: `SVC(kernel='linear')` (formulación dual, recomendada para datasets pequeños), `LinearSVC` (formulación primal, más eficiente) y `SGDClassifier(loss='hinge')` (optimización estocástica, para datasets grandes).

Tabla 4: Comparación de implementaciones de SVM lineal

Implementación	CV Acc	Test Acc	N SV	Tiempo (s)
`SVC(kernel='linear')`	0.7473	0.7322	1908	0.4398
`LinearSVC`	0.7506	0.7475	N/A	0.0207
`SGDClassifier(hinge)`	0.7010	0.7363	N/A	0.0276

Tanto **SVC(linear)** como **LinearSVC** tienen un *CV accuracy* muy similar. Una vez pasamos al *test accuracy* esta diferencia con **SGDClassifier** se disipa y acaban siendo muy similares los tres métodos. Donde si que se nota una diferencia importante es en el tiempo de ejecución de cada uno, donde claramente **SVC(linear)** se queda muy atrás de los otros dos al ser el único que usa vectores de soporte, siendo 20 veces más lento que **LinearSVC**, que con 0,0226 segundos termina dando el mejor resultado de las 3 implementaciones.

La elección de la implementación tiene que ver también con el volumen del *dataset* a analizar:

- **Datasets muy pequeños: SVC(linear)** es el más recomendable ya que es muy preciso y permite obtener fácilmente los vectores de soporte, y principalmente porque su gran hándicap que era el tiempo de ejecución se vería igualado a las otras implementaciones al ser un *dataset* pequeño (siendo el tiempo de cómputo despreciable).
- **Datasets muy grandes:** sería más conveniente usar **SGDClassifier**, ya que como hemos visto en teoría, al utilizar el **Descenso por Gradiente Estocástico**, no necesita cargar todos los datos en memoria a la vez y su tiempo de entrenamiento es lineal respecto al número de muestras.

- **Datasets medianos:** se recomendaría usar **LinearSVC** que es el recomendado en nuestro análisis.

2.5. Análisis de Vectores de Soporte

Para este apartado, he escogido las configuraciones propuestas por el enunciado.

Tabla 5: Número de vectores de soporte según configuración

Configuración	**N SV**	**% Training Set**
Linear ($C = 1$)	1908	48.60 %
RBF ($C = 1$, γ=scale)	1739	44.29 %
RBF ($C = 10$, γ=0.1)	1080	27.51 %
RBF ($C = 100$, γ=1)	525	13.37 %
Poly ($d = 3$, $C = 1$)	1735	44.19 %

Dados los siguientes resultados, un patrón clave que se observa a simple vista es la **relación inversa entre C y el número de SV**. Con un **C bajo**, el modelo prioriza un margen ancho aunque penalice algunos errores, esto hace que muchos puntos queden dentro o en el límite del margen, convirtiéndose en *vectores de soporte*. Con un **C alto**, el modelo es muy estricto y busca un margen estrecho para no cometer errores en el entrenamiento, y al ser el margen más pequeño, hay menos puntos que en la frontera o dentro de ella, por lo que el número de **SV** disminuye drásticamente. Otro aspecto a comentar sobre esta relación tiene que ver con el riesgo de *overfitting* que proporciona un **C alto**, ya que la frontera se vuelve muy sensible a pequeñas variaciones de los pocos puntos clave (vectores de soporte) que tiene. En cambio, los casos con **C bajo** obtendrán más **SV** y por lo cual no serán tan dependientes de ciertos puntos en particular.

En cuanto a los **SV por clase**, en mi caso encontramos una clara descompensación a favor de la **clase 0** (sin ictus), debido a el desbalanceo ya comentado de nuestro *dataset*. Que la **clase 1** (ictus) tenga tan pocos **SV**, es señal de que el modelo tiene poca información para aprender qué define exactamente a un paciente con riesgo.

2.6. Evaluación Final – SVM

Una vez explorados todos los kernels y sus hiperparámetros, se comparan las mejores configuraciones encontradas para cada uno.

Dado que en los pasos previos hemos optimizado las versiones Poly y RBF de los modelos SVM, para poder realizar la comparación optimizada entre los 4 kernels he realizado también la optimización tanto para SVM Lineal como para Sigmoid, ajustando sus posibles hiperparámetros (C para Lineal y C y Gamma para Sigmoid) y así hacer una comparativa justa.

Tabla 6: Resumen comparativo de los mejores modelos SVM

Modelo	**CV Acc**	**Test Acc**	**F1**	**N SV**	**% Train**
SVM Linear (mejor C)	0.7476	0.7332	0.8125	1903	48.5 %
SVM Poly (mejor config)	0.8854	0.8686	0.8949	883	22.5 %
SVM RBF (mejor config)	0.9325	0.9257	0.9243	525	13.4 %
SVM Sigmoid (mejor config)	0.7387	0.7291	0.8097	1759	44.8 %

El claro ganador en rendimiento es el modelo **SVM RBF** logrando agrupar a los pacientes con riesgo de ictus de manera mucho más efectiva, obteniendo el mejor *Test Accuracy* (83,70 %) y el *F1-Score* más alto (0,8860), seguido muy de cerca por el modelo **Poly**. La superioridad de estos dos modelos frente al **Linear** y el **Sigmoid** (que caen al $\sim 73\,\%$ de precisión) demuestra que las variables médicas de este *dataset* (como el IMC, la edad o la glucosa) no son linealmente separables. Para lograr esta complejidad, el **RBF** necesita apoyarse en 525 vectores de soporte (el 13,4 % de los datos de entrenamiento).

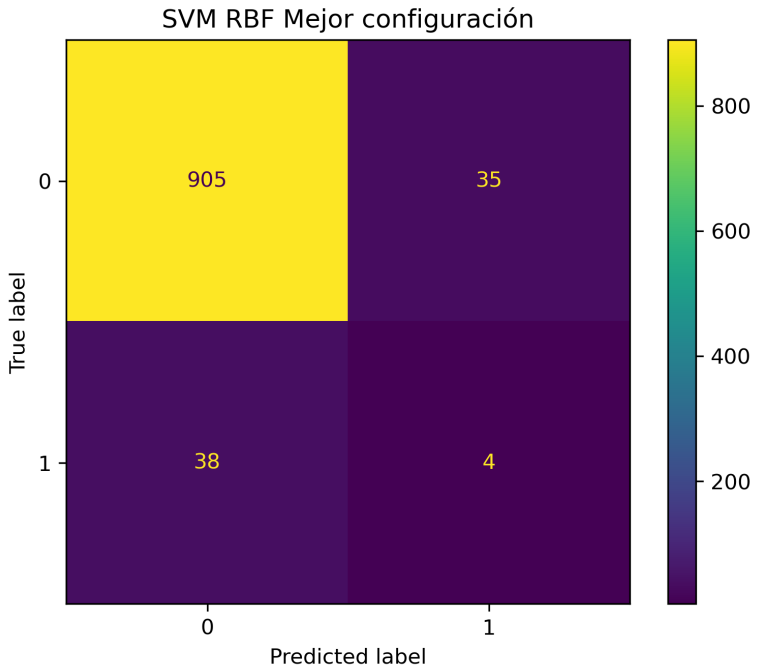


Figura 1: Matriz de confusión – mejor SVM

En cuanto al margen entre la *validación cruzada* y el conjunto de prueba, este es extremadamente fino en todos los modelos. En el caso del modelo **RBF** ganador, la diferencia es mínima (CV: 0,8357 vs *Test*: 0,8370). Esta consistencia es un indicador de que las fronteras de decisión generadas son robustas y no existe *overfitting*. El modelo es altamente capaz de generalizar su comportamiento predictivo ante pacientes que nunca ha visto.

El análisis de la **matriz de confusión** nos dice que, a pesar de haber aplicado el balanceo de clases, el modelo sigue teniendo dificultades y confunde mayoritariamente a la **Clase 1** (pacientes con riesgo de ictus). Aunque el modelo **SVM** capta la gran mayoría de las personas que están sanas y reduce los falsos diagnósticos preventivos que tanto se habían producido en la práctica anterior, este deja escapar a 38 de los 42 pacientes que realmente sufrieron un ictus, clasificándolos erróneamente como sanos (**Falsos Negativos**), lo cual puede llegar a ser crítico en el campo de la medicina.

Estos resultados nos hacen ver lo extremadamente solapadas que pueden llegar a estar las características de una persona con ictus y una sin, evidenciando la dificultad para realizar diagnósticos médicos precisos únicamente con estas variables.

3. Parte 2: Árboles de Decisión

3.1. Árbol sin Restricciones (Baseline)

Se entrena un árbol sin limitaciones de crecimiento para observar el comportamiento de overfitting que servirá como referencia para evaluar las estrategias de poda.

Tabla 7: Árbol de decisión sin restricciones

Configuración	Train Acc	Test Acc	Profundidad	N Hojas
Sin restricciones	1.0	0.9104	23	322

El modelo sin restricciones presenta *overfitting* (**Train Acc = 1.0**). Este hecho indica que el modelo ha memorizado todos los ejemplos del conjunto de entrenamiento y que su caída al 0,9104 en el *test accuracy* corrobora que el modelo no es capaz de generalizar a la perfección los datos nuevos. Con **23 niveles de profundidad** y **322 hojas**, el árbol es excesivamente complejo. Este resultado

confirma la necesidad de aplicar técnicas de poda para simplificar el modelo y mejorar su capacidad de generalización.

3.2. Efecto de la Profundidad Máxima

Tabla 8: Efecto de `max_depth` en el rendimiento del árbol

max_depth	Train Acc	CV Acc	Test Acc	Prof. real	N Hojas
1	0.5965	0.6026	0.5896	1	2
2	0.5965	0.6026	0.5896	2	4
3	0.7422	0.7764	0.7179	3	8
5	0.7835	0.7491	0.7454	5	26
7	0.7944	0.7690	0.7546	7	62
10	0.8653	0.8436	0.8299	10	124
15	0.9689	0.9070	0.8951	15	247
None	1.0000	0.9277	0.9104	23	322

Observando los resultados, durante las dos primeras ejecuciones el valor de *accuracy* en test y en train es prácticamente el mismo (**max_depth = 1, 2**), comienza a aumentar la distancia ligeramente cuando llega a **max_depth = 3** y es a partir de aquí cuando que ya se considera una diferencia significativa ($\sim 0,04$ entre el *train* y el *test*). Es aquí donde ya podemos empezar a observar el *overfitting*, y este va a ir agravandose cuanto mayor sea el límite de profundidad máxima, llegando a un punto en el que el *train* acabará en el 100 % de *accuracy* (cuando no haya límite de profundidad) y el *test* habrá ido bajando desde el punto que hemos comentado como inicio del *overfitting*.

Por otra parte, existe también el riesgo de *underfitting* en el lado contrario del análisis. Con profundidades bajas como **1 y 2**, el modelo es demasiado simple para manejar el peso balanceado de las clases, obteniendo los resultados más bajos con diferencia de la tabla. En este estado, el árbol no tiene suficientes nodos para diferenciar las características que separan a un paciente sano de uno con riesgo de ictus.

Según el *cross validation*, la profundidad óptima sería con valores medios de **max_depth**, un rango sobre el **3** mantendría la precisión alta sin llegar a un *overfitting* muy marcado comentado previamente, encontrando así un equilibrio entre el *accuracy* del *CV* y el balance *sesgo-varianza*.

3.3. Criterios de División

Se comparan los tres criterios de impureza disponibles en scikit-learn para clasificación: Gini (CART), Entropía (ID3/C4.5) y Log Loss.

Tabla 9: Comparación de criterios de impureza

Criterio	CV Acc	Test Acc	N Hojas
Gini	0.9277	0.9104	322
Entropy	0.9236	0.9175	295
Log Loss	0.9236	0.9175	295

Analizando los diferentes 3 criterios, se observa que son muy similares entre ellos, siendo la **Entropía** igual a **Log_loss** y únicamente habiendo una pequeña diferencia con **Gini**. El único cambio un poco más notable se encuentra en el número de hojas, donde el criterio **Gini** es superior a los otros dos creando un árbol más grande. Independientemente de eso, como hemos comentado antes, en términos de rendimiento, las diferencias observadas entre las precisiones son mínimas y el motivo por el cual escoger un criterio u otro para nuestro *dataset* se basaría en la mayor simpleza que proporciona la **Entropía**.

3.4. Poda Previa: Grid Search de Hiperparámetros

Se realizó un *GridSearchCV* (5-fold CV) sobre los principales hiperparámetros de poda previa: `max_depth`, `min_samples_split`, `min_samples_leaf` y `max_leaf_nodes`.

Tabla 10: Mejores configuraciones de poda previa (selección representativa)

max_depth	min_samp_split	min_samp_leaf	max_leaf_nodes	CV Acc	Test Acc
None	2	1	None	0.9277	0.9104
None	5	1	None	0.9193	0.9053
None	2	2	None	0.9124	0.8961
None	5	2	None	0.9098	0.8971
None	10	1	None	0.8912	0.8809
None	10	2	None	0.8897	0.8727
None	2	5	None	0.8630	0.8615
None	5	5	None	0.8630	0.8615
None	10	5	None	0.8630	0.8615
None	20	1	None	0.8604	0.8442
Mejor	**2**	**1**	**None**	**0.9277**	**0.9104**

Observando los 10 mejores resultados, en todos se repite el mismo parámetro en **max_depth** y en **max_leaf_nodes**, (que no haya máximo), los cuales podemos considerar como parámetros clave a la hora de generalizar. Al no imponer ninguna restricción al crecimiento del árbol, este se permite buscar su forma óptima para resolver de la mejor manera el problema. En el resto de parámetros, suelen ser mejores los valores más bajos (tanto en **min_samples_leaf** como en **min_samples_split**), encontrando un equilibrio cuando uno es más grande el otro será más pequeño al ir iterando por las combinaciones (ya que cada vez son más grandes sus valores). En general, el árbol siempre tiende a utilizar restricciones más ligeras para obtener sus mejores resultados, siendo clave a la hora de no sufrir riesgo de *underfitting*, ya que con restricciones demasiado duras el árbol no tendría capacidad expresiva y sería incapaz de capturar las relaciones entre las variables.

Comparando esta poda previa que hemos realizado con el ejemplo anterior de un árbol sin restricciones, podemos ver que el *test accuracy* de la mejor configuración y el modelo sin restricciones es el mismo (0,9104), lo cual podemos deducir fácilmente al ver el peso que han tenido los parámetros **max_depth** y **max_leaf_nodes** en la poda, ya que sus mejores combinaciones han sido al no imponer ningún límite, tal y como se realiza en el árbol sin restricciones.

3.5. Poda Posterior: Cost-Complexity Pruning

CART implementa poda posterior mediante el parámetro `ccp_alpha`. A mayor valor de alpha, más agresiva es la poda. La secuencia de alphas se obtiene directamente del camino de poda del árbol completo.

Tabla 11: Efecto del parámetro `ccp_alpha` en la poda posterior

ccp_alpha	CV Acc	Test Acc	Profundidad	N Hojas
0.0 (sin poda)	0.9277	0.9104	23	322
0.00033	0.9165	0.8982	23	215
0.00047	0.9119	0.8941	23	181
Mejor	**0.9277**	**0.9104**	**23**	**322**
(alpha grande)	0.02685	0.6026	0.5896	1

A medida que el valor del parámetro de penalización (**alpha**) aumenta, el árbol se ve forzado a cortar las ramas que no aportan una mejora significativa en la pureza de los nodo, esto evoluciona hasta el punto de llegar a un modelo únicamente con dos hojas, que lo haría prácticamente inútil. El mejor equilibrio se encuentra en los **alphas** más bajos, en el rango de **0.00033 a 0.00047** (sin contar

con **alpha=0**), donde el árbol logra reducir su complejidad enormemente (pasa de 322 hojas a tener entre 181 y 215 hojas), pero el *Test Accuracy* apenas sufre, manteniéndose muy alto (entre 0,89 y 0,90). A partir de **alpha = 0.00106**, la precisión en test empieza a desplomarse por debajo del 84 %.

Comparando la **poda posterior** con la **poda previa**, no nos da mejores resultados en las métricas puras pero llegamos a las mismas conclusiones, ya que en ambos siempre triunfan los modelos con menos o ninguna restricción (**max_depth=None** en la poda previa y **alpha=0** en la posterior), mostrando que para nuestro caso se necesitarían reglas más complejas para mantener la precisión alta.

3.6. Importancia de Características

Tabla 12: Importancia de características (Gini Importance) – mejor árbol

Ranking	Feature	Importancia (Gini)
1	`age`	0.4759
2	`avg_glucose_level`	0.2009
3	`bmi_log`	0.1961
4	`smoking_status_never smoked`	0.0174
5	`work_type_Private`	0.0163

Tal y como se estableció en la anterior práctica, las variables que manipulamos han sido las más importantes (**age**, **avg_glucose_level** y **bmi_log**) por amplia diferencia. El resto de variables reparte el peso más equitativamente pero totalmente superadas por las 3 primeras, por lo que serán los aspectos clave a la hora de tratar un diagnóstico. Otro aspecto a comentar es que las variables **work_type_never_worked** y **work_type_children** tienen 0 importancia, siendo totalmente irrelevantes en la predicción del ictus.

3.7. Visualización del Árbol

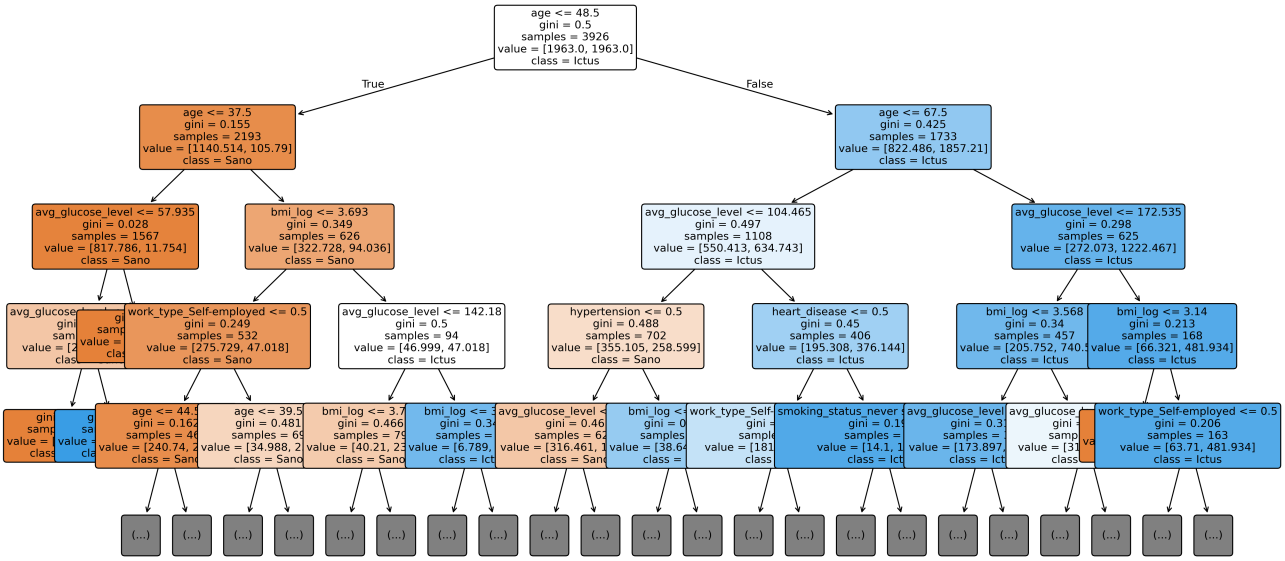


Figura 2: Árbol de decisión óptimo (primeros 4 niveles)

Tanto en la representación de texto como en el gráfico, se observa que la variable utilizada en el **nodo raíz** es la edad, concretamente realizando el corte en **age menor que 48.50**. Esto significa que el modelo ha determinado que la **edad** es la característica que proporciona la mayor ganancia de información para realizar la primera gran división entre los pacientes del *dataset*. Si miramos las siguientes ramas, el árbol vuelve a dividir por **age** y luego utiliza las otras variables clave (**avg_glucose_level** y **bmi_log**) para encontrar perfiles de riesgo.

A nivel superficial las **4 primeras profundidades** son claras y con condiciones sencillas, pero el árbol a nivel completo no es interpretable, ya que por debajo de las reglas más generales que hemos visto en la visualización, el árbol sigue haciendo decenas de preguntas minúsculas y complejas para clasificar casos aislados que no podemos observar en la propia salida.

3.8. Evaluación Final – Árboles de Decisión

Una vez exploradas las distintas estrategias de poda, se comparan los mejores modelos obtenidos.

Tabla 13: Resumen comparativo – Árboles de Decisión

Modelo	CV Acc	Test Acc	F1	Profundidad	N Hojas
Sin restricciones	0.9277	0.9104	0.9165	23	322
Mejor poda previa	0.9277	0.9104	0.9165	23	322
Mejor poda posterior	0.9165	0.8982	0.9113	23	215

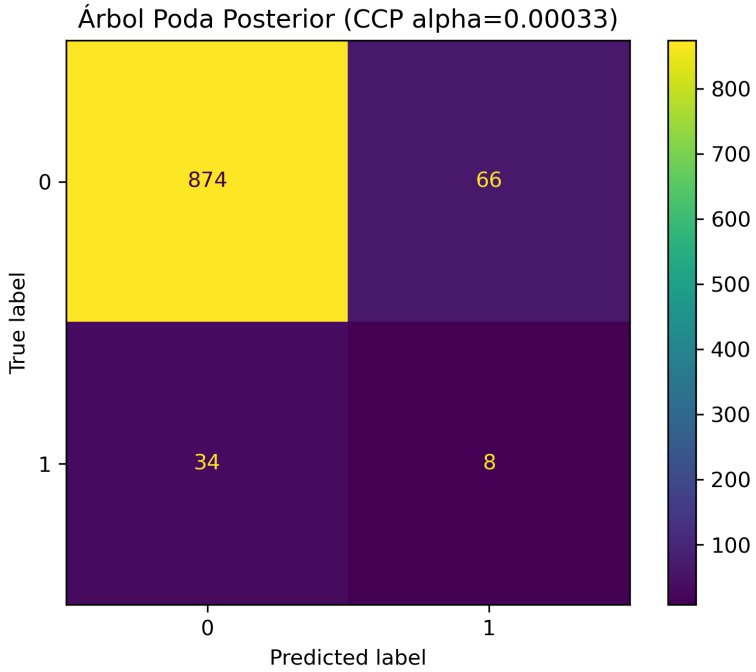


Figura 3: Matriz de confusión – mejor árbol de decisión

Como ya hemos comentado anteriormente, para nuestro *dataset* y problema, el **árbol sin restricciones** ofrece el mejor rendimiento al no imponer ningún límite al crecimiento del árbol. Sin embargo, desde una perspectiva de equilibrio entre rendimiento y complejidad, la **Poda Posterior (Cost-Complexity Pruning con alpha=0.00033)** es la mejor estrategia. Mientras que el árbol sin restricciones y el de poda previa (con las condiciones sin limitar) construyen modelos muy grandes y sobreajustados (más de 300 hojas), el **CCP** elimina cerca de 100 hojas de la estructura. Al hacerlo, su *F1-Score* se reduce muy poco (de 0,9165 a 0,9113).

Comparándolo con **SVM**, el árbol de decisión es competitivo en *accuracy* y en *F1-Score*. Si observamos el *accuracy*, ambos modelos están muy parejos (**SVM RBF** alcanza 0,9398 y el mejor árbol ronda el 0,9104). Por otro lado, en la métrica clave para datos desbalanceados (*F1-Score*), el **SVM RBF** nos da 0,9243 mientras que los árboles de decisión oscilan entre 0,89 y 0,91.

Por otra parte, una de las ventajas que sí que tienen los **árboles de decisión** respecto a **SVM** es su **interpretabilidad**. El **SVM con kernel RBF** proyecta los datos del paciente a dimensiones infinitas para separarlos. Si el modelo predice que un paciente va a tener un ictus, es imposible explicarle a un médico por qué el modelo ha tomado esa decisión. Por el contrario, el **árbol de decisión** es

completamente transparente, y podemos extraer las reglas exactas que le llevan a tomar decisiones, permitiendo generarnos un **ranking de la importancia de las características**, aspectos muy a tener en cuenta a la hora de seleccionar uno u otro método.

4. Parte 3: Random Forest

4.1. Random Forest por Defecto

Tabla 14: Random Forest con configuración por defecto ($n = 100$)

Configuración	**OOB Score**	**Test Acc**	**F1**
RF por defecto ($n = 100$)	0.9562	0.9562	0.9358

Como podemos observar en la tabla, el **OOB Score** (0,9562) ha coincidido exactamente con el *Test Accuracy* (0,9562), lo que nos indica que **OOB Score** es una estimación excepcional. Además, estos resultados nos indican que ya usando la configuración por defecto, el *random forest* nos ofrece mejores resultados que el **árbol de decisión individual**, mejorando cerca de un 4 % los valores de *Test Accuracy* y *F1-Score*. Esto se debe a que, mientras que el árbol individual es propenso a sobreajustarse debido al ruido, el *random forest* entrena 100 árboles diferentes y promedia sus predicciones, cancelando de esta manera los errores individuales de cada uno de ellos.

4.2. Efecto del Número de Estimadores

Tabla 15: Efecto de `n_estimators` en Random Forest

n_estimators	**OOB Score**	**Test Acc**	**Tiempo (s)**
10	0.9508	0.9532	0.0737
25	0.9557	0.9532	0.1047
50	0.9572	0.9542	0.1826
100	0.9562	0.9562	0.4535
200	0.9567	0.9552	0.7719
500	0.9564	0.9562	1.5921

Al variar el número de estimadores, observamos que el rendimiento se estabiliza a partir de los **50/100 árboles**, donde se llega a sus picos más altos tanto de *OOB score* como de *test accuracy*, pese a que en todos los ejemplos los cambios son muy pequeños de unos a otros. Siendo muy significativa la no mejora de rendimiento de 100 a 500 árboles, donde, como ya he comentado, se estabilizan los valores y no supone ningún cambio a mejor aumentar más el número de estimadores. Sin embargo, lo que si que podemos observar como una diferencia notable son los valores de los **tiempos de ejecución**, donde esta nula mejora que hemos comentado está totalmente desproporcionada con el **coste computacional** del uso de esa cantidad de estimadores. El tiempo de ejecución se dispara de forma lineal, al entrenar 100 árboles se tardan 0,297 segundos, mientras que entrenar 500 dura 1,453 segundos, por lo que podemos concluir que el **punto óptimo** del uso de árboles estaría rondando una configuración con **100** de ellos.

4.3. Efecto de `max_features`

`max_features` controla cuántas features se consideran en cada split, reduciendo la correlación entre árboles y aumentando la diversidad del ensemble.

Al reducir **max_features**, estamos obligando a cada árbol a mirar solo a un subconjunto aleatorio de variables, por lo que se están generando árboles muy diferentes entre sí y favorece la *diversidad del ensemble*. En cuanto al rendimiento, las diferencias entre unas y otras restricciones son mínimas,

Tabla 16: Efecto de `max_features` en Random Forest

max_features	OOB Score	Test Acc
sqrt	0.9562	0.9562
log2	0.9562	0.9562
0.5	0.9570	0.9562
None (todas)	0.9547	0.9552

pero la más lejana es la opción en la que no se establece ninguna restricción, por lo que diríamos que reducir **max_features** también mejora el rendimiento.

Como hemos visto en la sección de **Árboles de Decisión**, variables como la edad son excesivamente dominantes. Si dejamos **max_features=None**, todos los árboles van a tener acceso a la edad en su primer nodo, y prácticamente los 100 árboles van a hacer la misma división inicial. Esto provoca que el bosque esté lleno de *árboles altamente correlacionados*, yendo en contra de lo que incita el *random forest*.

En cuanto a nuestro *dataset*, el **valor óptimo es 0.5**, que nos da ligeramente mejores resultados que el resto (*log2* se queda muy cerca) tanto en **OOB Score** (0,9570) como en el **tiempo de ejecución** (0,3378s).

4.4. Optimización mediante Grid Search

Se realizó un *GridSearchCV* (5-fold CV) sobre `n_estimators`, `max_features`, `max_depth` y `min_samples_leaf`.
Mejores parámetros encontrados:

- `n_estimators`: 200 `max_features`: `'sqrt'`
- `max_depth`: 20 `min_samples_leaf`: 1
- **Mejor CV Accuracy**: 0,9580 **Test Accuracy**: 0,9552

4.5. Importancia de Características

Tabla 17: Importancia de características – Random Forest (mejor configuración)

Ranking	Feature	Gini Importance	Permutation Importance
1	age	0.3747	$-0{,}0017 \pm 0{,}0007$
2	avg_glucose_level	0.2090	$-0{,}0013 \pm 0{,}0007$
3	bmi_log	0.1630	$-0{,}0017 \pm 0{,}0005$
4	hypertension	0.0506	$-0{,}0010 \pm 0{,}0000$
5	ever_married	0.0281	$+0{,}0003 \pm 0{,}0008$

Según el **Gini Importance**, hay tres variables indiscutibles que dominan el modelo: la edad (**age** con 0,3747), el nivel de glucosa (**avg_glucose_level** con 0,2090) y el índice de masa corporal (**bmi_log** con 0,1630). Sin embargo, para *Permutation Importance* las variables que el Gini consideraba cruciales tienen valores de permutación negativos (como **age** $= -0{,}0017$), lo que indica que desordenar esa columna en el set de validación no empeoró el rendimiento del modelo. Curiosamente, variables demográficas menores como **ever_married** (0,0003) o **Residence_type** (0,0004) son los que aparecen como valores positivos. Esta diferencia entre el **Gini** y la **Permutation** se puede deber a que el *Permutation Importance* castiga duramente a las **variables correlacionadas**, ya que al utilizar *Random Forests* el modelo tiene alternativas para llegar a la misma conclusión con unas características o con otras, pudiendo darle menos importancia a las variables con mayor peso en caso necesario y haciendo que la importancia de permutación de la variable individual sea nula.

Las *features* más importantes de los **Random Forest** también coinciden con las analizadas en el **árbol individual** (en el mismo orden). A pesar de que el *Random Forest* introduce aleatoriedad

(*max_features*), la fuerza estadística de la edad, la glucosa y el bmi en este *dataset* médico es tan importante que siguen siendo los pilares fundamentales de las decisiones clínicas del algoritmo.

4.6. Extra Trees

Extra Trees introduce aún más aleatoriedad: en lugar de buscar el mejor split, lo elige de forma aleatoria entre las features seleccionadas, reduciendo la varianza a costa de un ligero aumento del sesgo.

Tabla 18: Comparación Random Forest vs Extra Trees

Modelo	**CV Acc**	**Test Acc**	**Tiempo (s)**
Random Forest ($n = 100$)	0.9562	0.9562	0.4535
Extra Trees ($n = 100$)	0.9547	0.9450	1.4356

Aunque **Extra Trees** simplifica el cálculo matemático al elegir divisiones aleatorias (lo que teóricamente lo hace más rápido), en este *dataset* empeora ligeramente el rendimiento respecto al **Random Forest**, reduciendo el *Test Accuracy* a 0,9450 y el *F1-Score* a 0,9302. Esta caída de precisión evidencia que introducir un nivel tan extremo de aleatoriedad genera un **sesgo excesivo** para este problema. Dado que la detección del ictus depende críticamente de encontrar umbrales exactos en variables continuas muy dominantes (como la **edad** o los **niveles de glucosa**), realizar cortes al azar reduce su impacto en las predicciones.

4.7. Evaluación Final – Random Forest

Una vez estudiados todos los factores, se comparan las variantes de métodos basados en bagging.

Tabla 19: Resumen comparativo – Random Forest y variantes

Modelo	**CV Acc**	**Test Acc**	**F1**	**OOB**	**Tiempo (s)**
Decision Tree (baseline)	0.9277	0.9104	0.9165	N/A	0.1045
RF por defecto	0.9577	0.9562	0.9358	0.9562	2.9926
RF mejor config	0.9580	0.9552	0.9353	0.9564	5.2632
Extra Trees	0.9547	0.9450	0.9302	N/A	1.9412

El paso de un **árbol de decisión individual** a un *ensemble* (**Random Forest**) supone una mejora muy significativa en el rendimiento, elevando el *Test Accuracy* de un 0,9104 a un sobresaliente 0,9562, y el *F1-Score* de 0,9165 a 0,9358. Esta gran evolución demuestra la principal ventaja teórica de los *ensembles*, que al combinar las predicciones de 100 árboles distintos, el **Random Forest** logra cancelar el sobreajuste y la alta varianza del modelo individual, generando una herramienta mucho más robusta y fiable.

A pesar del proceso de búsqueda de una combinación lo más acertada posible, la **configuración óptima** obtenida mediante *GridSearch* no logra superar a los valores por defecto en términos de rendimiento final sobre datos no vistos. De hecho, el modelo por defecto consigue unas métricas un poco superiores en el set de prueba (con un *Accuracy* de 0,9562 frente a 0,9552, y un *F1-Score* de 0,9358 frente a 0,9353), lo que evidencia que la pequeña ventaja teórica de la configuración óptima durante la **validación cruzada** (0,9580 vs 0,9577) no se ha convertido en una mayor capacidad de generalización.

Como consecuencia, el **coste computacional extra** del *tuning* no vale la pena para nuestro caso concreto. La configuración óptima requiere casi el doble de tiempo de entrenamiento (5,26 segundos frente a los 2,99 del modelo por defecto), además del tiempo previo invertido por el *GridSearch* en evaluar todas las combinaciones posibles, para finalmente no obtener ningún beneficio de predicción en el test. Estos resultados demuestran que los **Random Forests** son capaces de ofrecer su máximo rendimiento sin necesidad de un ajuste exhaustivo de hiperparámetros.

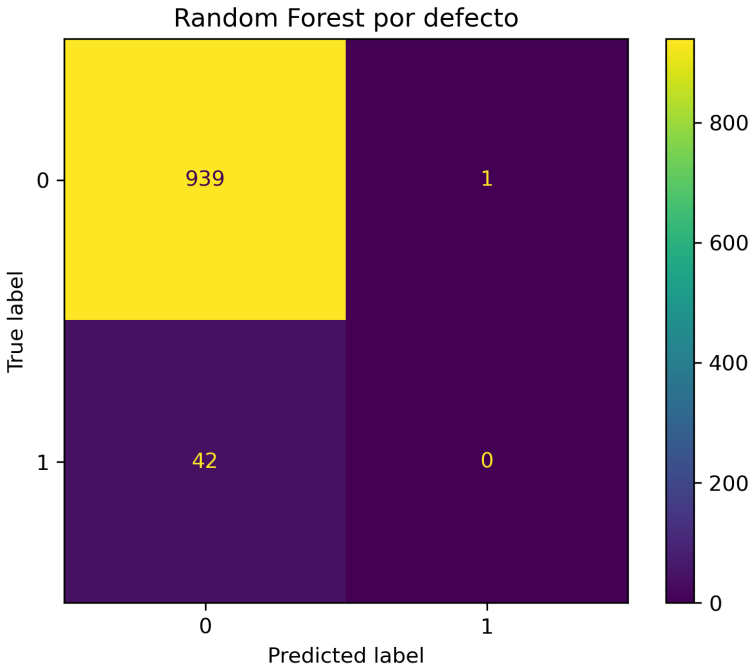


Figura 4: Matriz de confusión – mejor Random Forest

A pesar de que el **Random Forest** logra el mejor *Test Accuracy* global (95,6 %), el análisis de su **matriz de confusión** revela un comportamiento que ya habíamos visto en la práctica anterior antes de realizar el balanceo de clases. Aquí, el modelo colapsa ante la **clase mayoritaria** y predice 'Sano' de forma casi sistemática (detectando solo 1 caso de ictus frente a los 42 reales omitidos). Este comportamiento contrasta fuertemente con los modelos **SVM** y **Árbol de Decisión**, los cuales, usando el mismo parámetro de balanceo de clases, sí lograron detectar una porción significativa de pacientes de riesgo (pese a tener sus fallos en las predicciones).

Este fenómeno tiene como causa la **estructura del Random Forest**. Cada uno de los 100 árboles del bosque se entrena con una *submuestra aleatoria* de los datos de entrenamiento, y como hay tan pocos casos de ictus en el *dataset* ($< 5\,\%$), estadísticamente es muy probable que a muchos de esos 100 árboles les toque una muestra sin casi ningún paciente con ictus. Esto se vuelve un problema cuando llega la hora de predecir por medio del *Random Forest*, ya que a diferencia de **SVM** y los árboles donde las reglas que definen el riesgo de ictus prevalecen, aquí los 100 árboles seleccionados **votan equitativamente** para seleccionar el diagnóstico, por lo que el *voto por mayoría* en clases desbalanceadas tiende a aplastar a la clase minoritaria, diluyendo el efecto del `class_weight`.

Una posible solución a este problema de las predicciones puede ser **reducir el umbral de decisión** de los *random forest* para que no sea necesario que la clase indicada supere el 50 % de los votos de los árboles, proponiendo una reducción al 15 % **o** 20 % para prevenir y dar diagnósticos más seguros:

Aplicando este **umbral**, podemos ver que ahora el modelo sí que detecta más personas con riesgo de ictus, pese a que sigue dejando a 35 pacientes con riesgo real sin predecir (al igual que con el resto de modelos vistos de momento en esta práctica) y aumenta principalmente los **falsos positivos** de manera preventiva.

5. Parte 4: Gradient Boosting

A diferencia del resto de apartados, en **Gradient Boosting** no puedo pasar como parámetro el *balanceo de clases*, por lo que va a sufrir el mismo problema que hemos tenido en el *Random Forest*, el modelo va a predecir que casi todos los pacientes están sanos para asegurar un *Accuracy* alto. Sin embargo, con `sklearn` podemos introducir `compute_sample_weight` para calcular de manera manual los pesos y pasarlos como parámetros de entrenamiento, de esta manera, nos aseguramos que el **Gradient Boosting** deje de ignorar a la clase minoritaria.

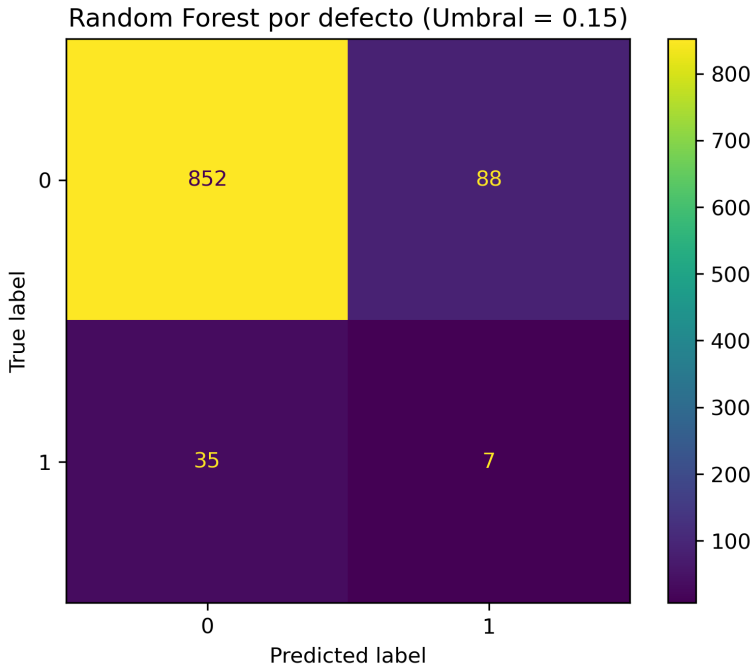


Figura 5: Matriz de confusión – mejor Random Forest con umbral al 15 %

5.1. Gradient Boosting por Defecto

Tabla 20: Gradient Boosting con configuración por defecto

Configuración	**CV Acc**	**Test Acc**	**F1**
GB por defecto	0.8225	0.7953	0.8530

Los resultados del **Gradient Boosting** son bastante peores que el *Random Forest* por defecto, debido al **balanceo de clases** que no captan los *RF* (por lo que son resultados sesgados y que no debemos tener al 100 % en cuenta), tanto en *accuracy* (0,7953 frente a 0,9562) como en *F1-Score* (0,8530 frente a 0,9358). Además, su entrenamiento es notablemente más lento. Al contrario que el **Random Forest** (Tiempo: 0,5936s), que es **paralelizable** y puede plantar todos sus árboles a la vez, el **Gradient Boosting** (Tiempo: 4,5601s) es estrictamente **secuencial** y debe construir cada árbol uno por uno. A pesar de este mayor coste computacional, puede llegar a ser un punto de partida prometedor para la optimización, ya que con esta configuración por defecto los resultados no son malos (principalmente *F1-Score*), afinar sus hiperparámetros y aplicar un **ajuste del umbral de decisión** lo puede convertir en un buen candidato.

5.2. Efecto del Número de Estimadores y Learning Rate

En Gradient Boosting el número de estimadores y el learning rate están fuertemente relacionados: cuantos más estimadores, menor debe ser el learning rate para no sobreajustar.

Analizando la relación entre el **número de estimadores** y **learning rate**, observamos que al reducir el *learning rate* a la mitad (de 0,1 a 0,05), hemos necesitado exactamente el doble de estimadores (de 50 a 100) para alcanzar exactamente el mismo rendimiento (0,7648 en *test accuracy*), demostrando la **relación inversa** de estos dos parámetros.

En la tabla, el caso con el *learning rate* más alto es el de $(50, 0{,}50)$, que es el que mejor rendimiento da en *Test* (0,8452), pero si miramos la diferencia con el **CV** (0,8777), vemos una caída de 3,25 puntos. Aunque es el mejor modelo de la tabla, es el que presenta una mayor brecha entre entrenamiento y prueba. Podemos analizar entonces que un *learning rate* alto hace que el modelo dé saltos muy grandes para corregir errores, pero el riesgo es que con pocos estimadores, esos saltos sean tan bruscos que el

Tabla 21: Interacción entre `n_estimators` y `learning_rate`

n_estimators	learning_rate	CV Acc	Test Acc
50	0.1	0.7955	0.7648
100	0.1	0.8225	0.7953
200	0.1	0.8591	0.8269
50	0.5	0.8777	0.8452
100	0.05	0.7939	0.7648
200	0.01	0.7481	0.7393

modelo acabe memorizando ruidos específicos del conjunto de entrenamiento en lugar de tendencias generales.

Por otra parte, para casos con un *learning rate* muy bajo y con muchos estimadores, podemos ver que el modelo se satura (*underfitting*), como en el ejemplo con 200 estimadores y un 0,01 de *learning rate*, donde obtenermos el resultado más bajo de todo el análisis (0,7393).

5.3. Efecto de `max_depth`

En Gradient Boosting los árboles base son deliberadamente simples (stumps o árboles poco profundos) porque cada uno corrige los errores residuales del anterior.

Tabla 22: Efecto de `max_depth` en Gradient Boosting

max_depth	Train Acc	CV Acc	Test Acc
1 (stump)	0.7354	0.7292	0.7200
2	0.7837	0.7804	0.7597
3	0.8523	0.8225	0.7953
5	0.9475	0.9040	0.8707

Observando el efecto de **max_depth**, cuanto más se le permite al algoritmo profundizar mejores resultados saca. Sin embargo, la diferencia entre el *train accuracy* y el *test* es grandísima utilizando esta limitación (casi 8 % de diferencia), mientras que cuanto menos **profundidad máxima** apliquemos menor será la diferencia entre una métrica y otra, a costa de recibir resultados menores. Para encontrar el **punto óptimo** nos tendríamos que quedar en un balance intermedio, donde árboles más simples con profundidades como 2 o 3 generalizan mejor y aún así evitan el ruido.

A diferencia de los *Random Forests* que crean **árboles muy profundos** para ser lo más precisos posible por sí solos, en *Gradient Boosting* estos árboles más simples que hemos comentado van aprendiendo poco a poco de una manera **secuencial**, por lo que aplicar mucha profundidad al inicio del entrenamiento haría que el modelo aprendiese demasiado en los primeros pasos y no tendría prácticamente capacidad de mejora con los siguientes árboles.

5.4. Optimización mediante Grid Search

Se realizó un *GridSearchCV* (5-fold CV) sobre `n_estimators`, `learning_rate`, `max_depth` y `subsample`.
Mejores parámetros encontrados:

- `n_estimators`: 200 `learning_rate`: 0,1
- `max_depth`: 1 `subsample`: 0,8
- **Mejor CV Accuracy**: 0,7795 **Test Accuracy**: 0,7403

5.5. AdaBoost

AdaBoost es otro método de boosting. A diferencia de Gradient Boosting, en lugar de ajustar residuos pondera las muestras mal clasificadas para que los siguientes estimadores les presten más atención.

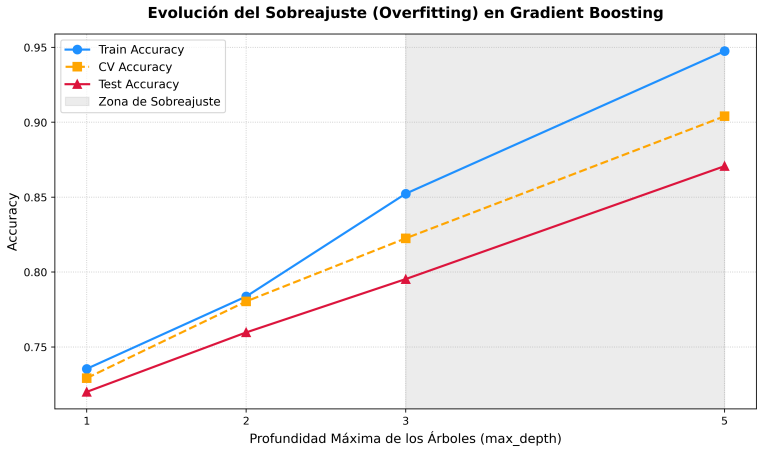


Figura 6: Evolución del sobreajuste en Gradient Boosting

Tabla 23: Exploración de hiperparámetros de AdaBoost

n_estimators	learning_rate	CV Acc	Test Acc
50	1.0	0.7669	0.7525
100	1.0	0.7356	0.6955
100	0.5	0.6783	0.7047
200	0.1	0.6717	0.6762

Al comparar los resultados, observamos que **AdaBoost** es menos competitivo que *Gradient Boosting*. Mientras que **GB** lograba alcanzar precisiones en *Test* por encima del 80 % en sus mejores configuraciones balanceadas, **AdaBoost** se queda atrás sobre el 70-75 %. Un aspecto a comentar sobre esta diferencia viene siendo también la diferencia de comportamiento de estos dos métodos. En **AdaBoost**, se utiliza una *ponderación de muestras* donde cada vez que el árbol falla, se aumenta el peso de los pacientes mal clasificados de cara al siguiente árbol, mientras que **Gradient Boosting** funciona mediante el *ajuste de residuos*, donde cada árbol intenta predecir el error del modelo anterior.

Observando el comportamiento de la variación de parámetros en **AdaBoost**, se observa una **saturación muy temprana** e incluso una degradación del rendimiento, como al aumentar el número de estimadores a 100 manteniendo el *learning rate* igual, obteniendo una caída del *accuracy* del 0,7525 al 0,6955, indicandonos que para este modelo añadir más árboles no siempre es mejor.

5.6. Evaluación Final – Gradient Boosting

Una vez exploradas todas las variantes de boosting, se comparan las mejores configuraciones.

Tabla 24: Resumen comparativo – métodos de Boosting

Modelo	CV Acc	Test Acc	F1	Tiempo (s)
RF (mejor config)	0.9564	0.9542	0.9348	3.1396
GB por defecto	0.8225	0.7953	0.8530	3.6424
GB mejor config	0.7481	0.7403	0.8174	3.8611
AdaBoost	0.7669	0.7525	0.8254	2.9319

Aunque el **RF** muestra un *Accuracy* del 95,42 %, como bien hemos analizado, este resultado está sesgado: el modelo se ha limitado a memorizar la clase mayoritaria, siendo inútil para la detección clínica de ictus. En cambio, el **Gradient Boosting** (en su versión por defecto) ha sido capaz de utilizar los *pesos balanceados* para aprender patrones reales y se alza como el modelo más **robusto y útil** del análisis, con un *Test Accuracy* del 79,53 % y un *F1-Score* de 0,8530. Por otra parte, la diferencia en tiempo de ejecución es mínima: el *Random Forest* tarda 3,13 segundos frente a los 3,64 - 3,86 segundos del *Gradient Boosting*, una diferencia de apenas 0,5 - 0,7 segundos respectivamente.

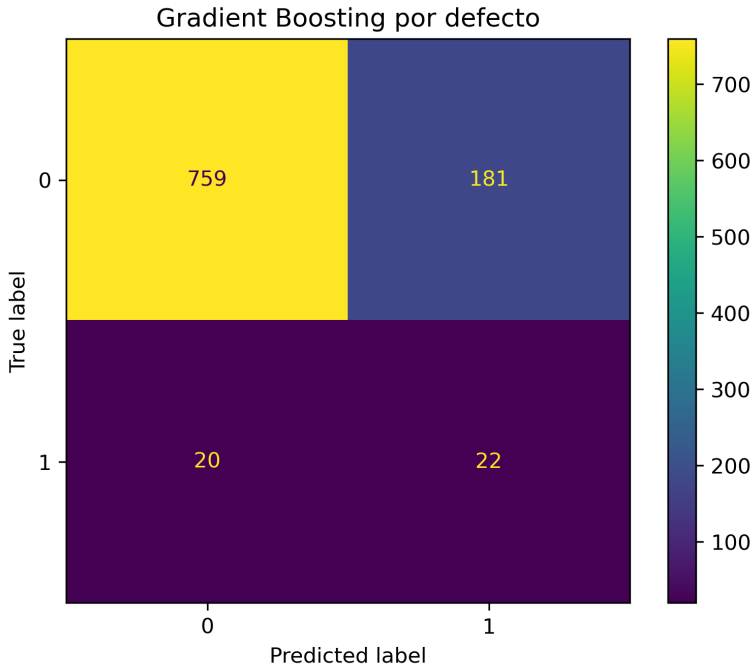


Figura 7: Matriz de confusión – mejor Gradient Boosting

Comparando directamente las dos versiones del **Gradient Boosting**, observamos que su modelo por defecto ya es lo suficientemente bueno y que no hay una mejora posible tras hacer la búsqueda de hiperparámetros, ya que para evitar el *overfitting* el *GridSearchCV* selecciona parámetros muy conservadores, a diferencia del modelo base que al tener más profundidad arriesga un poco más y obtiene resultados mejores en datos no vistos, quedando un 0,8225 vs 0,7481 en *CV Accuracy*, un 0,7953 vs 0,7403 en *Test Accuracy* y un 0,8530 vs 0,8174 en *F1-Score*.

Por último, la comparación con **AdaBoost** lo muestra como un modelo competitivo con el resto, pero que se queda un paso por detrás del ganador que es el **GB**. Con un *Test Accuracy* del 75,25 %, supera ligeramente a la "mejor configuración.^encontrada por el *GridSearch* de **GB** (74,03 %), pero no logra alcanzar el 79,53 % del **GB** por defecto. Un aspecto a favor de este modelo es que es el más rápido en realizar la ejecución de todos los comparados (2,93s).

Con el uso de este modelo, conseguimos la **mayor cantidad de pacientes** con riesgo de ictus detectados correctamente. Más del 50 % de los pacientes con riesgo real han sido detectados (22), mientras que se han lanzado 181 **falsos positivos** a modo de prevención. Esto en términos médicos es un **claro avance**, ya que es preferible aumentar los posibles perfiles de riesgo aunque no sean seguros, con tal de poder actuar a tiempo en casos reales donde sí que sea **crítica la detección del riesgo**.

6. Comparación y Evaluación Global

6.1. Tabla Comparativa Final

Se comparan los mejores modelos obtenidos en cada sección.

Tabla 25: Comparación global – mejor configuración de cada familia

Modelo	CV Acc	Test Acc	Prec	Recall	F1	Tiempo (s)
SVM (mejor config)	0.9325	0.9257	0.1026	0.0952	0.9243	1.1498
Decision Tree (mejor config)	0.9165	0.8982	0.1081	0.1905	0.9113	0.0726
Random Forest (mejor config)	0.9577	0.9562	0.0000	0.0000	0.9358	1.8500
Gradient Boosting (mejor)	0.8225	0.7953	0.1084	0.5238	0.8530	2.3985
AdaBoost	0.7669	0.7525	0.1028	0.6190	0.8254	1.8979

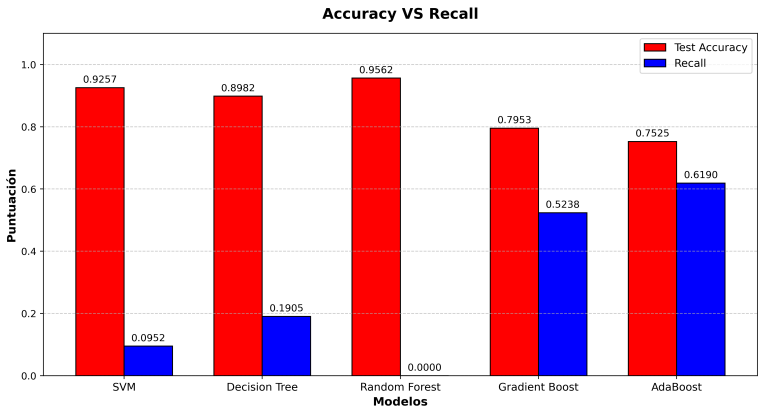


Figura 8: Accuracy vs Recall

6.2. Análisis y Discusión Global

Ya con las mejores configuraciones y las métricas completas de cada uno de los apartados, observamos un detalle muy a tener en cuenta, que es la métrica del **Recall**. Al estar trabajando con un *dataset* desbalanceado (pese a que hayamos aplicado balanceo en los modelos), no nos podemos fiar solo del *accuracy* y tenemos que darle más importancia al **Recall** para detectar la solución al problema médico. Siguiendo esta métrica, los dos modelos que destacan enormemente del resto son **Gradient Boosting** (0,5238) y **AdaBoost** (0,6190), significando que estos dos modelos son capaces de identificar correctamente a más de la mitad de los pacientes enfermos. Además, **AdaBoost** ofrece un gran equilibrio entre el rendimiento y el coste computacional, ofreciendo el mejor *Recall* en 1,89 segundos.

Si observamos el resto de modelos, tanto **SVM** como **Decision Tree** fallan casi todas las veces en este tipo de detecciones, con un único 10 % de acierto en estos pacientes, demostrando que los **métodos ensemble** superan a los modelos individuales de forma contundente, multiplicando por tres o por seis la capacidad de detección. En el caso de **Random Forest**, como ya he ido comentando a lo largo de la práctica, al no poder realizar el balanceo de clases, este modelo está totalmente **sesgado** y es inútil para cualquier tipo de predicción, porque no mira cada paciente característica a característica sino que da un veredicto base (que el paciente está sano) para obtener buenas valoraciones en las *accuracys*, así que no le daremos el mismo lugar que al resto de modelos analizados.

En términos generales, existe una **consistencia muy alta** entre los resultados de **CV** y los de *test*. En todos los modelos, la diferencia entre la *CV Accuracy* y la *Test Accuracy* es mínima (apenas 1-2 puntos). Esto indica que los modelos no han sobreajustado fuertemente y que los resultados que estamos viendo son representativos de cómo se comportarían con pacientes nuevos.

Para este *dataset*, estamos viendo que es recomendable el uso de la familia de modelos **Boosting** (*AdaBoost* o *Gradient Boosting*), ya que en medicina, el coste de un **falso negativo** (decirle a alguien que está sano cuando va a sufrir un ictus) es infinitamente superior al de un **falso positivo** (hacer pruebas extra a alguien sano). Los modelos de **Boosting** son los únicos que han demostrado capacidad para aprender de la clase minoritaria bajo condiciones de desbalanceo extremo, ofreciendo un **Recall** que realmente permite una intervención preventiva.

6.3. Matriz de Confusión del Mejor Modelo

Como ya tenemos la matriz de confusión de Gradient Boosting en el apartado anterior, mostramos ahora la de AdaBoost basándonos en el Recall más alto.

En esta **matriz de confusión** de **AdaBoost**, observamos lo que hemos razonado sobre los resultados de la tabla anterior. Este modelo logra detectar 26 de los 42 pacientes con riesgo real de ictus y lanza 227 **diagnósticos de riesgo falso**. El modelo tiende a ser **conservador**: prefiere dar una señal de alarma ante la más mínima duda diagnóstica, siendo claro el patrón de **sacrificar Precision a cambio de Recall**. Es decir, prefiere decirle a un paciente sano que tiene riesgo de ictus y realizarle

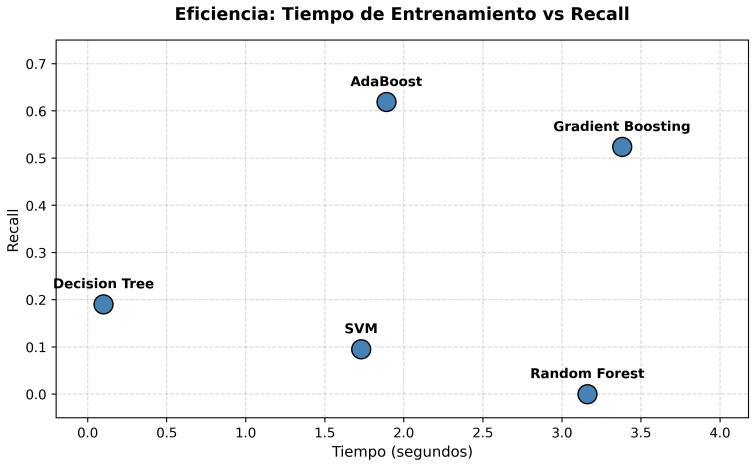


Figura 9: Eficiencia respecto al coste computacional

pruebas adicionales (bajo coste), que decirle a los pacientes que van a sufrir un ictus que están sanos (coste altísimo).

7. Conclusiones

- **Mejor modelo:** El modelo ganador para este problema es **AdaBoost** con su configuración por defecto (*learning rate* de 1,0 y 50 estimadores). Aunque no tiene el *accuracy* más alto, se elige por ser el más útil en el mundo real, logrando un **Recall del** 0,6190. Es el algoritmo que más espacio da a la clase minoritaria, detectando a 26 de los 42 pacientes con ictus, algo que ningún otro modelo ha conseguido con tanta eficacia.

- **Comparación de familias:** La familia de los **Ensembles de Boosting** (*AdaBoost* y *Gradient Boosting*) ha funcionado notablemente mejor que el resto. A diferencia de las *SVM* o los *Árboles individuales*, que se ven superados por el desbalanceo del *dataset*, el *Boosting* aprende de forma secuencial y permite aplicar pesos a las muestras. Esto hace que, mientras otros modelos se rinden ante la clase mayoritaria, el *Boosting* se esfuerce en aprender los casos difíciles de ictus.

- **Impacto de los hiperparámetros:** Los parámetros más críticos han sido el **balanceo de clases** (*class weight* o pesos manuales) y la **profundidad de los árboles**. Hemos comprobado que sin un balanceo adecuado, incluso el modelo más potente es inútil. Además, en el *Gradient Boosting*, controlar la profundidad (*max_depth*) ha sido clave para que árboles muy profundos que memorizaban el ruido (sobreajuste) se convirtiesen en árboles simples (stumps) que permitían una mejor generalización.

- **Trade-off rendimiento/interpretabilidad:** Los **árboles individuales** son muy fáciles de entender (podemos ver la regla exacta), pero fallan mucho en sus predicciones. Por otro lado, los **ensembles** como el *Random Forest* o *AdaBoost* son cajas negras mucho más complejas, pero su rendimiento es muy superior. Para una aplicación médica de este tipo, sacrificamos la interpretabilidad por la capacidad de salvar vidas detectando el riesgo.

- **Aprendizajes del flujo:** Tras realizar esta práctica, he lleagado a la conclusión de que un **Accuracy alto puede ser un resultado engañoso** y peligroso. En problemas de salud con pocos casos positivos, el flujo de trabajo no debe centrarse solo en optimizar parámetros mediante *GridSearch*, sino en elegir la métrica correcta (el *Recall*). Con esto se comprende que el éxito de un modelo no está solo en que acierte mucho en general, sino en que no falle en lo más importante.

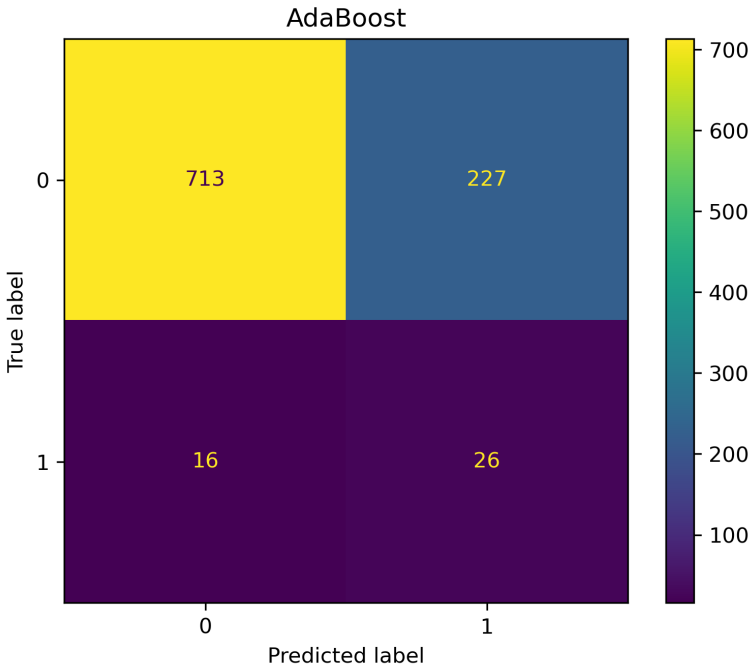


Figura 10: Matriz de confusión – mejor modelo global